

# ACTIVIDADES PRÁCTICAS DE APROPIACIÓN

Sesión 1 - Fundamentos y primeras pruebas con Pytest

Módulo II: Automatización de pruebas unitarias con Pytest

## GUÍA PARA EL APRENDIZ

|                     |                                     |
|---------------------|-------------------------------------|
| Programa o ficha    | Desarrollo de Software / QA Testing |
| Aprendiz            | Jorge Alejandro Torres Paez         |
| Instructor          | Jeysson Contreras                   |
| Fecha de desarrollo | 30 de Julio de 2026                 |

### Importante

Esta guía no contiene soluciones. Su propósito es orientar el desarrollo de las actividades y permitir que cada aprendiz construya, ejecute y analice sus propias pruebas.

**Duración sugerida: 2 horas 15 minutos**

## 1. Presentación de la sesión

En esta sesión se realiza el primer acercamiento práctico a Pytest. El trabajo se enfoca en reconocer cómo se descubren las pruebas, ejecutar la suite desde la terminal, interpretar resultados y organizar pruebas sencillas con el patrón Arrange-Act-Assert.

### Resultados de aprendizaje de la sesión

- Configurar y ejecutar una suite básica con Pytest.
- Reconocer las convenciones para descubrir archivos y funciones de prueba.
- Interpretar los estados PASSED, FAILED y los mensajes básicos de error.
- Aplicar el patrón Arrange-Act-Assert en funciones sencillas.
- Documentar los hallazgos obtenidos durante la ejecución de las pruebas.

### Recursos necesarios

- Computador con Python 3.12 o una versión compatible instalada.
- Editor de código, preferiblemente Visual Studio Code.
- Terminal o consola de comandos.
- Entorno virtual del proyecto activado.
- Pytest instalado en el entorno de desarrollo.

### Orientaciones generales

- Lea todas las instrucciones antes de modificar los archivos.
- Ejecute los comandos desde la carpeta raíz del proyecto.
- No cambie el código de producción salvo que una instrucción lo solicite.
- Registre los resultados de cada ejecución, incluso cuando una prueba falle.
- No copie pruebas terminadas: construya cada caso a partir del comportamiento esperado.
- Use nombres descriptivos para archivos y funciones de prueba.

### Comando de referencia

Para ejecutar la suite completa utilice: `python -m pytest -v`

## 2. Actividad 1.1 - Laboratorio de descubrimiento de pruebas

|                          |                                                                       |
|--------------------------|-----------------------------------------------------------------------|
| <b>Propósito</b>         | Reconocer cómo Pytest identifica archivos y funciones de prueba.      |
| <b>Duración sugerida</b> | 45 minutos.                                                           |
| <b>Modalidad</b>         | Trabajo en parejas.                                                   |
| <b>Producto</b>          | Tabla de descubrimiento, archivos corregidos y registro de ejecución. |

### Situación de aprendizaje

Se recibe un proyecto con varias operaciones matemáticas y tres archivos que pretenden contener pruebas. Algunos nombres cumplen las convenciones de Pytest y otros no. La misión es ejecutar el proyecto, observar qué descubre Pytest y justificar el resultado.

### Estructura inicial del proyecto

```
laboratorio-descubrimiento/
├── app/
│   ├── __init__.py
│   └── operations.py
└── tests/
    ├── operations_test.py
    ├── test_operations.py
    └── pruebas_matematicas.py
```

### Código inicial: app/operations.py

```
def add(number_a: float, number_b: float) -> float:
    return number_a + number_b

def subtract(number_a: float, number_b: float) -> float:
    return number_a - number_b

def multiply(number_a: float, number_b: float) -> float:
    return number_a * number_b

def divide(number_a: float, number_b: float) -> float:
    if number_b == 0:
        raise ValueError("No es posible dividir entre cero")
    return number_a / number_b
```

## Archivos de prueba entregados

Cree los siguientes archivos exactamente con los nombres y contenidos indicados. No los corrija todavía.

### tests/test\_operations.py

```
from app.operations import add

def test_add_two_positive_numbers():
    result = add(5, 3)
    assert result == 8
```

### tests/operations\_test.py

```
from app.operations import subtract

def test_subtract_two_numbers():
    result = subtract(10, 4)
    assert result == 6
```

### tests/pruebas\_matematicas.py

```
from app.operations import multiply

def comprobar_multiplicacion():
    result = multiply(4, 5)
    assert result == 20
```

## Procedimiento

1

### Ejecute la suite

Desde la raíz del proyecto ejecute `python -m pytest -v`. No modifique todavía ningún nombre.

2

### Observe el reporte

Identifique cuántas pruebas fueron recolectadas, cuáles fueron ejecutadas y qué estado obtuvo cada una.

3

### Complete la tabla

Registre si cada función fue descubierta y explique la razón con base en las convenciones de Pytest.

4

### Corrija lo necesario

Renombre únicamente los archivos o funciones que no estén siendo reconocidos. Decida los nuevos nombres antes de aplicarlos.

5

### Ejecute nuevamente

Compruebe si Pytest descubre ahora todas las pruebas y compare el nuevo reporte con la primera ejecución.

6

### Realice ejecuciones selectivas

Ejecute una prueba específica y luego seleccione pruebas mediante una palabra clave usando la opción `-k`.

## Registro de descubrimiento

| Archivo                | Función                       | ¿Fue ejecutada? | Justificación                                                   |
|------------------------|-------------------------------|-----------------|-----------------------------------------------------------------|
| test_operations.py     | test_add_two_positive_numbers | Sí              | Cumple el formato correcto: archivo test_*.py y función test_*. |
| operations_test.py     | test_subtract_two_numbers     | Sí              | Cumple el formato correcto: archivo *_test.py y función test_*. |
| pruebas_matematicas.py | comprobar_multiplicacion      | No              | El archivo y la función no empiezan con 'test_'.                |

## Preguntas de análisis

### 1. ¿Cuántas pruebas fueron recolectadas en la primera ejecución?

Se recolectaron 2 pruebas.

### 2. ¿Qué convenciones de nombres identificó para los archivos?

Pytest reconoce archivos que empiezan con 'test\_' o terminan en '\_test.py'.

### 3. ¿Qué convención identificó para las funciones de prueba?

El nombre de la función debe empezar con 'test\_'.

### 4. ¿Qué cambios realizó y por qué?

Se renombró 'pruebas\_matematicas.py' a 'test\_pruebas\_matematicas.py' y la función a 'test\_comprobar\_multiplicacion', para que Pytest pudiera reconocerlos.

### 5. ¿Qué diferencia observó entre ejecutar toda la suite y ejecutar una sola prueba?

Ejecutar toda la suite revisa todo el proyecto; ejecutar una sola prueba es más rápido cuando se quiere revisar un caso puntual.

### 6. ¿Para qué puede resultar útil la opción -k?

Sirve para ejecutar solo las pruebas cuyo nombre contiene una palabra clave, sin correr toda la suite.

## Evidencia requerida

- ☒ Tabla de descubrimiento diligenciada.
- ☒ Registro o captura de la primera ejecución.
- ☒ Registro o captura de la ejecución posterior a las correcciones.
- ☒ Lista de los nombres que fueron modificados.

✓ Respuesta a las preguntas de análisis.

**Criterio de logro**

La actividad se considera completada cuando el aprendiz puede explicar, sin memorizar comandos, por qué Pytest reconoce unos elementos y omite otros.

### 3. Actividad 1.2 - Construcción de pruebas con Arrange-Act-Assert

|                          |                                                                             |
|--------------------------|-----------------------------------------------------------------------------|
| <b>Propósito</b>         | Organizar pruebas sencillas utilizando las etapas Arrange, Act y Assert.    |
| <b>Duración sugerida</b> | 60 minutos.                                                                 |
| <b>Modalidad</b>         | Trabajo individual.                                                         |
| <b>Producto</b>          | Archivo de pruebas creado por el aprendiz y análisis de una prueba fallida. |

#### Situación de aprendizaje

Una tienda necesita verificar los cálculos de una factura. El aprendiz debe crear pruebas para el subtotal, el impuesto y el total, separando claramente la preparación de los datos, la ejecución de la función y la comprobación del resultado.

#### Código inicial: app/invoice.py

```
def calculate_subtotal(unit_price: float, quantity: int) -> float:
    return unit_price * quantity

def calculate_tax(subtotal: float, tax_percentage: float) -> float:
    return subtotal * tax_percentage / 100

def calculate_invoice_total(
    unit_price: float, quantity: int, tax_percentage: float
) -> float:
    subtotal = calculate_subtotal(unit_price, quantity)
    tax = calculate_tax(subtotal, tax_percentage)
    return subtotal + tax
```

#### Orientación conceptual

Arrange prepara los datos; Act ejecuta una sola acción principal; Assert compara el resultado obtenido con el comportamiento esperado. Evite mezclar varias reglas diferentes en la misma prueba.



## Retos prácticos

| Reto | Función a probar        | Datos de entrada                                         | Resultado esperado | Nombre de la prueba          |
|------|-------------------------|----------------------------------------------------------|--------------------|------------------------------|
| 1    | calculate_subtotal      | Precio unitario: 25.000<br>Cantidad: 4                   | 100.000            | test_calculate_subtotal      |
| 2    | calculate_tax           | Subtotal: 100.000<br>Impuesto: 19 %                      | 19.000             | test_calculate_tax           |
| 3    | calculate_invoice_total | Precio unitario: 50.000<br>Cantidad: 2<br>Impuesto: 19 % | 119.000            | test_calculate_invoice_total |

## Procedimiento orientado

1

### Cree el archivo de pruebas

Dentro de la carpeta tests, cree un archivo cuyo nombre cumpla las convenciones de Pytest.

2

### Importe las funciones

Importe únicamente las funciones necesarias desde app.invoice.

3

### Nombre cada prueba

Use nombres que indiquen la unidad probada, el comportamiento esperado y, cuando sea necesario, la condición evaluada.

4

### Organice la prueba

Separe visualmente Arrange, Act y Assert. Utilice una acción principal por caso.

5

### Ejecute y revise

Ejecute la suite con el modo detallado y compruebe que cada caso aparece de forma independiente.

6

### Introduzca un fallo controlado

Cambie temporalmente uno de los resultados esperados por un valor incorrecto. Ejecute nuevamente y analice el mensaje de Pytest. Después restaure el valor correcto.

## Plantilla de planificación de cada prueba

| Prueba | Arrange: datos requeridos            | Act: función y resultado              | Assert: comparación esperada |
|--------|--------------------------------------|---------------------------------------|------------------------------|
| Reto 1 | unit_price=25000, quantity=4         | calculate_subtotal(25000, 4)          | result == 100000             |
| Reto 2 | subtotal=100000, tax_rate=19         | calculate_tax(100000, 19)             | result == 19000.0            |
| Reto 3 | unit_price=50000, quantity=2, tax=19 | calculate_invoice_total(50000, 2, 19) | result == 119000.0           |

## Análisis del fallo controlado

### 1. ¿Qué prueba falló?

'test\_calculate\_tax'.

### 2. ¿Qué valor obtuvo realmente la función?

El valor correcto: 19000.0.

### 3. ¿Qué valor esperaba la prueba?

Por error, esperaba 20000.

### 4. ¿En qué archivo y línea se informó el fallo?

En 'tests/test\_invoice.py', línea 25.

### 5. ¿Cómo ayudó el mensaje de Pytest a localizar el problema?

Pytest mostró la comparación 19000.0 == 20000, dejando claro dónde estaba la diferencia.

## Evidencia requerida

- ☒ Archivo de pruebas desarrollado por el aprendiz.
- ☒ Tres casos de prueba ejecutados de manera independiente.
- ☒ Uso visible y coherente de Arrange, Act y Assert.
- ☒ Registro de la ejecución exitosa.
- ☒ Registro del fallo controlado y análisis escrito.
- ☒ Restauración final de la suite en estado exitoso.

## Criterios de revisión formativa

| Criterio                                                         | Cumple                              | Observaciones |
|------------------------------------------------------------------|-------------------------------------|---------------|
| El archivo y las funciones cumplen las convenciones de Pytest.   | <input checked="" type="checkbox"/> |               |
| Los nombres de las pruebas describen el comportamiento evaluado. | <input checked="" type="checkbox"/> |               |
| Las etapas Arrange, Act y Assert están claramente diferenciadas. | <input checked="" type="checkbox"/> |               |
| Cada prueba verifica un comportamiento principal.                | <input checked="" type="checkbox"/> |               |
| El aprendiz interpreta correctamente el reporte de fallo.        | <input checked="" type="checkbox"/> |               |

## 4. Cierre de la sesión

---

Antes de finalizar, revise el trabajo realizado y confirme que puede repetir el procedimiento sin depender de una solución proporcionada por el instructor.

### Lista de comprobación del aprendizaje

- ☒ Activé el entorno virtual antes de ejecutar Pytest.
- ☒ Ejecuté las pruebas desde la raíz del proyecto.
- ☒ Identifiqué las convenciones de nombres de archivos y funciones.
- ☒ Pude ejecutar toda la suite y una prueba específica.
- ☒ Creé pruebas propias para las funciones de facturación.
- ☒ Organicé los casos con Arrange, Act y Assert.
- ☒ Interpreté correctamente un fallo deliberado.
- ☒ Restauré la suite y confirmé que todas las pruebas pasan.
- ☒ Organicé las evidencias solicitadas para revisión.

### Reflexión final

#### 1. ¿Qué fue lo más importante que aprendió sobre el descubrimiento de pruebas?

R/ Seguir bien los nombres de archivos y funciones es lo que permite que Pytest las encuentre solo, sin configurarlas a mano.

---

#### 2. ¿Qué diferencia existe entre ejecutar una función manualmente y comprobarla mediante una prueba automatizada?

R/ La prueba automatizada da siempre el mismo resultado, es más rápida y no depende de que una persona revise a mano cada vez.

---

#### 3. ¿Qué parte de Arrange-Act-Assert necesita seguir practicando?

R/ Preparar datos más complejos (Arrange), usando fixtures y simulando partes externas del sistema.

---

#### 4. ¿Cómo podría aplicar estas pruebas a una regla de negocio de una aplicación FastAPI?

R/ Separando la lógica de negocio en funciones simples, que se puedan probar con Pytest sin depender de la API.

---

### Preparación para la siguiente sesión

Conserve el proyecto y las pruebas creadas. En la siguiente sesión se trabajarán excepciones, escenarios negativos y análisis de valores de frontera.